Alexandria University
Faculty of Engineering
Computer and Systems Program
Artificial Intelligence Course

Assignment 1

Using Informed and Uninformed Search Algorithms to Solve the 8-Puzzle
BFS | DFS | Iterative DFS | A* (Manhattan & Euclidean)

Team members

Name	ID
اسلام محمد فاروق محمود بلبع	24010108
محمود حسين سالم محمد علام	24010666

Assigned: 10/10/2026 **Due:** 23/10/2026

1. Introduction

The 8-puzzle is a 3×3 board with eight numbered tiles and one blank (written as 0). A move slides a tile into the blank, which is the same as moving the blank Up, Down, Left or Right. The goal is the board 012/345/678. This report describes our C++ solver that uses BFS, DFS, Iterative DFS and A^* (with Manhattan and Euclidean heuristics), and compares them by path cost, nodes expanded, search depth and running time.

Component	Definition
State	A 3×3 arrangement of the tiles 0-8. Only $9!/2 = 181,440$ of the 9! arrangements are reachable from any start.
Actions	Move the blank Up, Down, Left or Right (when it stays on the board).
Goal test	The board equals 012/345/678.
Path cost	Every move costs 1, so cost = number of moves.

Results at a glance: BFS, Iterative DFS and A^* always found the shortest path; DFS found very long ones (up to 59,123 moves). On a 26-move board, A^* with Manhattan distance expanded 1,719 nodes, compared with 158,791 for BFS and 1,837,357 for Iterative DFS. Manhattan distance beat Euclidean distance (1,719 vs. 6,598 nodes).

2. Implementation

The code is C++17. Every algorithm is a class that works on one shared State type.

File	Purpose
State.hpp	The State (board, neighbour generation, goal test), its hash function, and the SearchResult structure with path reconstruction.
BFS.hpp, DFS.hpp, Iterative_DFS.hpp, A_star.hpp	One class per search algorithm.
main.cpp	Reads the algorithm number and the board, runs the search, measures the time with std::chrono, and writes solution.json.
server.py, index.html, app.js, style.css	Optional web page that runs the program and animates the solution.
makefile	Builds with g++-03 (output/main).

To run: `make`, then `./output/main`. Enter the algorithm number (1 BFS, 2 DFS, 3 Iterative DFS, 4 A^* Manhattan, 5 A^* Euclidean) and the nine tiles row by row.

3. Data Structures

Structure	Use
State	A 3×3 array of numbers. neighbors() finds the blank and returns the child boards with their move names, always in the order Up, Down, Left, Right.
StateHash	Packs the 9 cells into one 64-bit number (4 bits per cell). Different boards always get different values, so there are no hash collisions.
SearchResult	Stores the move list, the list of boards, cost, nodes expanded and search depth.
Frontier	std::queue (BFS), std::stack of (state, depth) (DFS), and std::priority_queue ordered by $f = g + h$ (A^*).
Visited tables	std::unordered_set (BFS); std::unordered_map holding the depth (DFS) or the best g (A^*) of each state.
parent_map	unordered_map from each state to (parent state, move). The path is rebuilt by walking back from the goal and reversing.

4. Algorithms

All algorithms generate children in the order Up, Down, Left, Right, never revisit a state they have already stored, and rebuild the path from parent_map. If the start already equals the goal, they return a path of cost 0.

4.1 Breadth-First Search (BFS)

BFS expands the shallowest node first using a FIFO queue. A set holds every state that was queued or expanded so none is queued twice. The goal is tested when a child is generated, which saves one extra level of expansion.

```
queue = [initial]; seen = {initial}
```

```
while queue not empty:
    curr = queue.pop_front(); nodes_expanded++
    for (child, move) in curr.neighbors():
        if child not in seen:
            parent[child] = (curr, move)
            if child == goal: return path(child)
            queue.push_back(child); seen.add(child)
```

- Complete and optimal (all moves cost 1). Time and memory are $O(b^{\wedge}d)$ (b = branching factor, d = solution depth); memory is its main weakness.

4.2 Depth-First Search (DFS)

DFS expands the deepest node first using a LIFO stack. Because children are pushed as Up, Down, Left, Right, the last one (Right, if possible) is expanded first. The goal is tested when a node is popped. With no depth limit (the default) each state is visited at most once. With a limit, nodes at the limit are not expanded, and a state is allowed to be re-opened if it is reached again at a smaller depth, so the limit does not hide solutions.

```
stack = [(initial, 0)]; depth[initial] = 0
while stack not empty:
    (curr, d) = stack.pop(); nodes_expanded++
    if curr == goal: return path(curr)
    if limit set and d >= limit: continue
    for (child, move) in curr.neighbors():
        if child not in depth or (limit set and d+1 < depth[child]):
            parent[child] = (curr, move); depth[child] = d+1
            stack.push((child, d+1))
```

- Complete here (finite space, repeated states filtered) but not optimal: it returns the first path it finds, which is usually very long.

4.3 Iterative Deepening DFS (IDS)

IDS runs the depth-limited DFS with limits 0, 1, 2, ... until a solution appears. The upper limit is 31, the largest number of moves any solvable 8-puzzle needs. Since each round repeats the earlier ones, the reported nodes_expanded is the sum over all rounds.

```
for limit = 0..31:
    result = DFS(initial, goal, limit)
    total_nodes += result.nodes_expanded
    if result found: return result (with nodes_expanded = total_nodes)
```

- Complete and optimal (the first successful limit is the shallowest solution depth). Time $O(b^{\wedge}d)$; the repeated work is the price for needing little memory in theory.

4.4 A^* Search

A^* picks the node with the smallest $f(n) = g(n) + h(n)$, where g is the number of moves so far and h estimates the moves left. A min-heap holds (f, state) pairs and a table stores the best

g of each state. A child is pushed whenever a cheaper path to it is found. The goal is tested when a node is popped, which makes the result optimal when h is admissible.

```
g[initial] = 0; heap.push((h(initial), initial))
while heap not empty:
    curr = heap.pop_min(); nodes_expanded++
    if curr == goal: return path(curr)
    for (child, move) in curr.neighbors():
        if child unseen or g[curr]+1 < g[child]:
            parent[child] = (curr, move); g[child] = g[curr]+1
            heap.push((g[child] + h(child), child))
```

- Complete and optimal with an admissible h (true for both heuristics below).

Note: when a state is improved, its old heap entry stays in the heap and may be expanded later. These duplicates are counted in nodes_expanded (see Section 7.4).

5. Heuristics

h(board) is the sum, over the eight tiles, of the distance from the tile's current cell to its goal cell (the goal cell of tile v is row $v/3$, column v%3).

- **Manhattan:** $|\Delta row| + |\Delta col|$ per tile. One move shifts one tile by one cell and changes this sum by exactly 1, so it never overestimates: it is admissible (and consistent).
- **Euclidean:** $V(\Delta row^2 + \Delta col^2)$ per tile. A straight line is never longer than the Manhattan path, so h_Euclidean $\leq$ h_Manhattan $\leq$ true cost: it is also admissible, but a weaker estimate.

Which is "more admissible"? Both are admissible, since neither overestimates. Manhattan is closer to the true cost (it dominates Euclidean), so A^* expands fewer nodes with it, as the results in Section 8 confirm.

6. Assumptions

- The goal is 012/345/678 and the blank is 0. The board is entered as nine numbers, row by row. Each move costs 1, so cost and search depth both equal the number of moves of the returned path.
- nodes_expanded counts nodes taken from the frontier. BFS tests children when they are generated, so the goal node is not counted; DFS and A^* count it; IDS sums all rounds.
- Running time covers only the search call (std::chrono, g++-O3). All times are single runs read from the web interface on our machine. Absolute values depend on the machine.
- The program does not validate the board or check solvability. An unsolvable board makes every algorithm exhaust the reachable states and return an empty path (Section 7.4).
- The fixed move order (Up, Down, Left, Right) decides which solution DFS finds and which of several equal-cost paths the other algorithms return.

7. Sample Runs

Each board below was run with all five variants. A separate script replayed every returned path on its start board and confirmed that it ends at the goal.

7.1 Assignment example: 2 5 4 / 3 0 1 / 6 7 8

Paths found (U = Up, D = Down, L = Left, R = Right):

- BFS, Iterative DFS, A^* (Manhattan), A^* (Euclidean): ULL
- DFS: LLDRRULLDRRULLDRRULLDRRULLDRUR (59,123 moves in total)

The board sequence of the shortest path (cost 3) is:

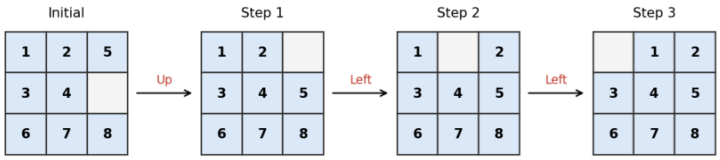


Figure 1: Board-by-board trace of the solution

Algorithm	Cost	Nodes expanded	Search depth	Running time (ms)
BFS	3	5	3	0.07
DFS	59,123	120,492	59,123	276.67
Iterative DFS	3	32	3	0.12
A^* (Manhattan)	3	4	3	0.07
A^* (Euclidean)	3	4	3	0.09

DFS also reaches the goal, but its path is far longer than the shortest one, because DFS does not look for the shortest path.

7.2 Board A: 7 2 4 / 5 0 6 / 8 3 1 (26 moves)

Paths found (U = Up, D = Down, L = Left, R = Right; algorithms with the same path are grouped):

- BFS: LURDDLURRULLDRRDLURULDDLUU
- DFS: RDLLURRDLLURRDLLURRDLLURRDLURD (48,404 moves in total)
- Iterative DFS, A^* (Manhattan), A^* (Euclidean): LURDRDLLURRDLLURRULLDR-RULL

Algorithm	Cost	Nodes expanded	Search depth	Running time (ms)
BFS	26	158,791	26	415.37
DFS	48,404	136,681	48,404	443.69
Iterative DFS	26	1,837,357	26	2,834
A^* (Manhattan)	26	1,719	26	5.31
A^* (Euclidean)	26	6,598	26	20.79

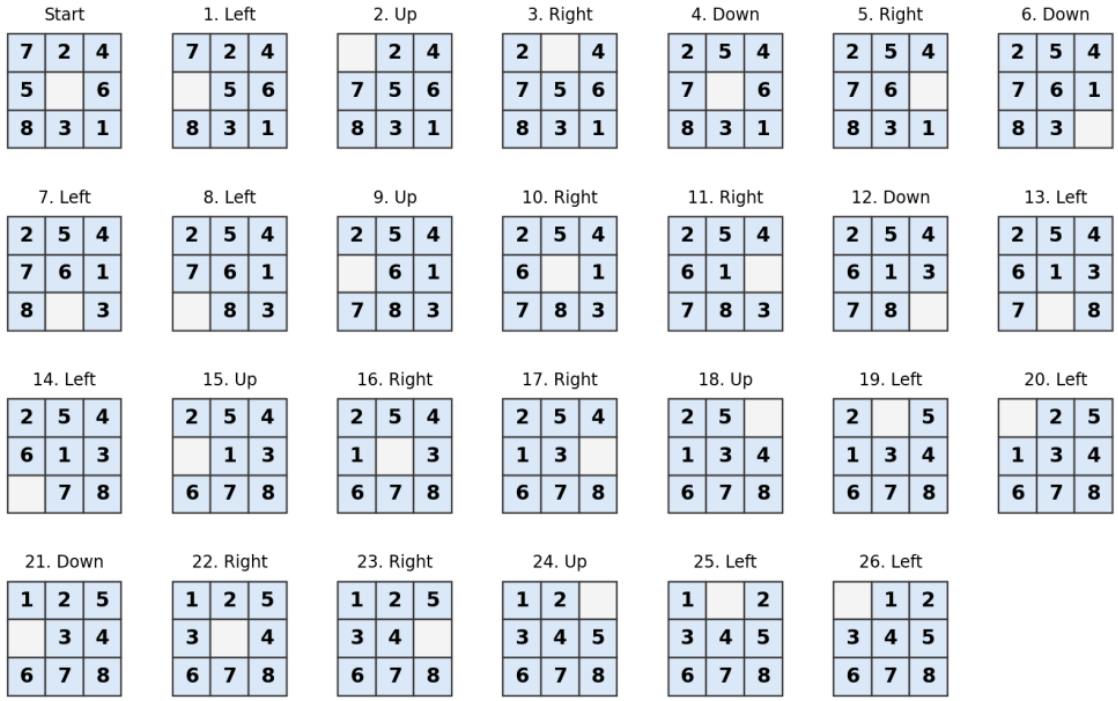


Figure 2: Full trace of the A^* (Manhattan) solution for Board A (each card is the board after that move)

7.3 Board B: 8 6 7 / 2 5 4 / 3 0 1 (27 moves)

Paths found:

- BFS, Iterative DFS, A^* (Manhattan), A^* (Euclidean): URDLUULDDRUULDDRURUL-DRDLULU
- DFS: RULLDRRULLDRRULLDRRULLDRRULLUR (52,089 moves in total)

Algorithm	Cost	Nodes expanded	Search depth	Running time (ms)
BFS	27	167,760	27	395.02
DFS	52,089	59,602	52,089	162.77
Iterative DFS	27	2,254,165	27	2,899
A^* (Manhattan)	27	4,206	27	13.66
A^* (Euclidean)	27	7,608	27	24.08

7.4 Special cases

Board D: 1 0 2 / 3 4 5 / 6 7 8 is one move from the goal (Left). BFS needs 1 expansion, but DFS expands 181,439 nodes, almost the whole state space, because its move order sends it down the wrong branches first.

Board E: 0 1 2 / 3 4 5 / 6 8 7 is the goal with two tiles swapped, which is unsolvable. All algorithms return an empty path, so cost and depth are shown as -:

Algorithm	Cost	Nodes expanded	Search depth	Running time (ms)
BFS	-	181,440	-	452.23
DFS	-	181,440	-	529.25
Iterative DFS	-	4,898,307	-	6,771
A^* (Manhattan)	-	189,685	-	441.07
A^* (Euclidean)	-	182,583	-	588.29

BFS and DFS expand exactly 181,440 nodes, which is 9!/2 the number of reachable states. A^* expands a few more (189,685 and 182,583) because of the duplicate heap entries mentioned in Section 4.4. IDS repeats its work for limits 0 to 31.

8. Comparison

Nodes expanded

Board	BFS	DFS	IDS	A^* Manh.	A^* Eucl.
Assignment example (3 moves)	5	120,492	32	4	4
Board D (1 move)	1	181,439	4	2	2
Board A (26 moves)	158,791	136,681	1,837,357	1,719	6,598
Board B (27 moves)	167,760	59,602	2,254,165	4,206	7,608

Running time (ms)

Board	BFS	DFS	IDS	A* Manh.	A* Eucl.
Assignment example (3 moves)	0.07	276.67	0.12	0.07	0.09
Board D (1 move)	0.04	344.45	0.06	0.04	0.03
Board A (26 moves)	415.37	443.69	2,834	5.31	20.79
Board B (27 moves)	395.02	162.77	2,899	13.66	24.08

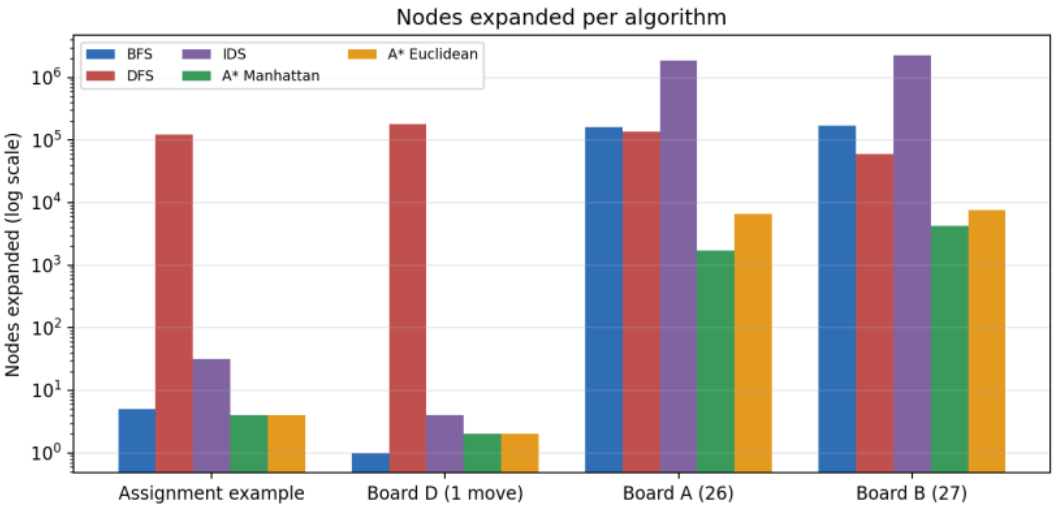


Figure 3: Nodes expanded (logarithmic scale)

Observations

- **Optimality:** BFS, IDS and both A^* variants always gave the same shortest cost. DFS gave far longer paths.
- **BFS vs. IDS:** on Board A, IDS expanded 11.6x more nodes than BFS (1,837,357 vs. 158,791) because it repeats earlier levels. Our version also keeps visited tables, so it gets little of IDS's usual memory advantage.
- **Informed vs. uninformed:** A^* with Manhattan expanded 92.4x fewer nodes than BFS on Board A (1,719 vs. 158,791) and 39.9x fewer on Board B.
- **Manhattan vs. Euclidean:** both are optimal, but Euclidean expanded 3.8x more nodes on Board A and 1.8x more on Board B. On the very easy boards they tie. Manhattan is the better heuristic.
- DFS is unpredictable: its node count depends on the move order, not on how near the goal is (59,602 nodes for Board B, but 181,439 for Board D).
- Times are small because the state space has only 181,440 states; the gap between algorithms would grow quickly for larger puzzles.

9. Extra Work

Besides the console search, we added a web visualiser: server.py runs the compiled program and returns solution.json, and the page animates the tiles, shows the statistics and offers a board-by-board trace of the whole solution. main.cpp also contains visualize_path(), which prints the solution as a grid in the console (currently commented out).

10. Conclusion

BFS, Iterative DFS and A^* find shortest solutions, while DFS finds a valid but very long one. A^* is by far the most efficient, and Manhattan distance is the better heuristic because it is admissible and closer to the true cost than Euclidean distance.